
Table of Contents

PREAMBLE	1
=====	=====
1. COMMENTING	1
=====	=====
=====	=====
2. DFT and DTFT	6
=====	=====
=====	=====
3. DFT and IDFT	14
=====	=====
=====	=====
4. DFT AND FFT	21
=====	=====
=====	=====
Functions provided for the lab	25

PREAMBLE

DO NOT REMOVE THE LINE BELOW

```
clear; close all;
```

```
=====
=====
```

1. COMMENTING

```
=====
=====
```

MAKE SURE FILE BELOW IS IN SAME DIRECTORY AS THIS FILE

```
type('eel3135_lab10_comment.m')
```

```
%% Question 1 Comment Code
clear all; clc; close all;
```

```
fs = 16000;
tt = 0:1/fs:0.01-1/fs;
```

```
w0 = 888*pi;
w1 = 3520*pi;
w2 = 14080*pi;
```

```

x = 0.5 + cos(w0*tt) + cos(w1*tt + pi/4) + cos(w2*tt + 2*pi/3);

N = [50,length(x),500,1000];

X1 = fft(x,N(1));
X2 = fft(x,N(2));
X3 = fft(x,N(3));
X4 = fft(x,N(4));

% Answer in your comments: How is the DFT (using the FFT algorithm)
% calculated when N is larger than the length of X?
% (Hint: read help FFT)
%
% When N is larger than the length of x, the FFT computes the DFT by
% zero-padding the signal x with (N - length(x)) zeros before performing
% the transform. This increases frequency resolution without changing the
% actual signal content.

% Answer on your comments: How is the DFT (using the FFT algorithm)
% calculated when N is smaller than the length of X?
%
% When N is smaller than the length of x, the FFT computes the DFT using
% only the first N samples of x (the signal is truncated to length N
% before performing the transform).

% Answer in your comments: Why would you not want to always make N as large
% as possible?
%
% You may not want to always make N as large as possible because
% making N very large increases computation time and memory usage. While it
% provides more frequency samples (higher frequency resolution), it does not
% reveal fundamentally new information about the signal—only a more detailed
% representation of the same spectrum. In some cases (when you have too
% little samples
% or sample at the wrong interval) increasing N can help you produce more
% accurate graphs, but you reach a point of diminishing returns, where
% increasing N does not help your graphs become much more accurate and only
% takes up computation time and storage.

b1 = -ceil((N(1)-1)/2):floor((N(1)-1)/2);
b2 = -ceil((N(2)-1)/2):floor((N(2)-1)/2);
b3 = -ceil((N(3)-1)/2):floor((N(3)-1)/2);
b4 = -ceil((N(4)-1)/2):floor((N(4)-1)/2);

% Answer in your comments: What frequencies do the peaks represent?
%
% The peaks represent the frequency components present in x[n]. The constant
% term (0.5) produces a DC component at 0 Hz. The cosine terms produce
% frequency components at  $\pm 444$  Hz,  $\pm 1760$  Hz, and  $\pm 7040$  Hz. Converting to
% normalized angular frequency using  $\omega = 2\pi(f/f_s)$  with  $f_s = 16000$  Hz:
%
% 444 Hz  $\rightarrow \omega = 2\pi(444/16000) = 0.0555\pi$  rad/sample
% 1760 Hz  $\rightarrow \omega = 2\pi(1760/16000) = 0.22\pi$  rad/sample
% 7040 Hz  $\rightarrow \omega = 2\pi(7040/16000) = 0.88\pi$  rad/sample

```

```

%
% Therefore, the peaks occur at 0,  $\pm 0.0555\pi$ ,  $\pm 0.22\pi$ , and  $\pm 0.88\pi$  rad/sample.
% In the DFT plots, these may appear at nearby values depending on N, since
% the DFT samples frequency at discrete points and may not land exactly on
% the true frequencies.

f1 = b1*fs/N(1);
f2 = b2*fs/N(2);
f3 = b3*fs/N(3);
f4 = b4*fs/N(4);

w1 = 2*pi/N(1)*b1;
w2 = 2*pi/N(2)*b2;
w3 = 2*pi/N(3)*b3;
w4 = 2*pi/N(4)*b4;

figure
subplot(221)
plot(b1,fftshift(abs(X1)))
hold on;
plot(b1,fftshift(abs(X1)), '.', 'markersize', 8)
hold off;
xlim([-N(1)/2 N(1)/2]); ylim([0 100]);
xlabel('Bin Number')
ylabel('Magnitude')
title(sprintf('Length %i DFT',N(1)))

subplot(222)
plot(b2,fftshift(abs(X2)))
hold on;
plot(b2,fftshift(abs(X2)), '.', 'markersize', 8)
hold off;
xlim([-N(2)/2 N(2)/2]); ylim([0 100]);
xlabel('Bin Number')
ylabel('Magnitude')
title(sprintf('Length %i DFT',N(2)))

subplot(223)
plot(b3,fftshift(abs(X3)))
hold on;
plot(b3,fftshift(abs(X3)), '.', 'markersize', 8)
hold off;
xlim([-N(3)/2 N(3)/2]); ylim([0 100]);
xlabel('Bin Number')
ylabel('Magnitude')
title(sprintf('Length %i DFT',N(3)))

subplot(224)
plot(b4,fftshift(abs(X4)))
hold on;
plot(b4,fftshift(abs(X4)), '.', 'markersize', 8)
hold off;
xlim([-N(4)/2 N(4)/2]); ylim([0 81]);
xlabel('Bin Number')

```

```
ylabel('Magnitude')
title(sprintf('Length %i DFT',N(4)))
```

```
figure
subplot(221)
plot(f1,fftshift(abs(X1)))
hold on;
plot(f1,fftshift(abs(X1)), '.', 'markersize', 8)
hold off;
xlim([-fs/2 fs/2]); ylim([0 81]);
xlabel('Frequency (Hz)')
ylabel('Magnitude')
title(sprintf('Length %i DFT',N(1)))
```

```
subplot(222)
plot(f2,fftshift(abs(X2)))
hold on;
plot(f2,fftshift(abs(X2)), '.', 'markersize', 8)
hold off;
xlim([-fs/2 fs/2]); ylim([0 81]);
xlabel('Frequency (Hz)')
ylabel('Magnitude')
title(sprintf('Length %i DFT',N(2)))
```

```
subplot(223)
plot(f3,fftshift(abs(X3)))
hold on;
plot(f3,fftshift(abs(X3)), '.', 'markersize', 8)
hold off;
xlim([-fs/2 fs/2]); ylim([0 81]);
xlabel('Frequency (Hz)')
ylabel('Magnitude')
title(sprintf('Length %i DFT',N(3)))
```

```
subplot(224)
plot(f4,fftshift(abs(X4)))
hold on;
plot(f4,fftshift(abs(X4)), '.', 'markersize', 8)
hold off;
xlim([-fs/2 fs/2]); ylim([0 81]);
xlabel('Frequency (Hz)')
ylabel('Magnitude')
title(sprintf('Length %i DFT',N(4)))
```

```
figure
subplot(221)
plot(w1,fftshift(abs(X1)))
hold on;
```

```
plot(w1,fftshift(abs(X1)), '.', 'markersize', 8)
hold off;
xlim([-pi pi]); ylim([0 81]);
xlabel('Normalized Frequency (rad/s)');
ylabel('Magnitude')
title(sprintf('Length %i DFT',N(1)))
```

```
subplot(222)
plot(w2,fftshift(abs(X2)))
hold on;
plot(w2,fftshift(abs(X2)), '.', 'markersize', 8)
hold off;
xlim([-pi pi]); ylim([0 81]);
xlabel('Normalized Frequency (rad/s)');
ylabel('Magnitude')
title(sprintf('Length %i DFT',N(2)))
```

```
subplot(223)
plot(w3,fftshift(abs(X3)))
hold on;
plot(w3,fftshift(abs(X3)), '.', 'markersize', 8)
hold off;
xlim([-pi pi]); ylim([0 81]);
xlabel('Normalized Frequency (rad/s)');
ylabel('Magnitude')
title(sprintf('Length %i DFT',N(3)))
```

```
subplot(224)
plot(w4,fftshift(abs(X4)))
hold on;
plot(w4,fftshift(abs(X4)), '.', 'markersize', 8)
hold off;
xlim([-pi pi]); ylim([0 81]);
xlabel('Normalized Frequency (rad/s)');
ylabel('Magnitude')
title(sprintf('Length %i DFT',N(4)))
```

```
% Answer in your comments: How does the length of the DFT affect the
% magnitude? Be sure to zoom in on the bases of the peaks in each DFT
% magnitude plot.
%
% As the DFT length N increases, the frequency resolution improves, which
% affects how the magnitude of the peaks appears. For smaller N, the true
% frequencies may not align with DFT bins, causing spectral leakage where
% energy is spread across multiple bins and peak magnitudes appear lower.
% As N increases, the DFT samples frequencies more densely, allowing the
% peaks to better align with their true frequencies and concentrate their
% energy, resulting in higher and more accurate peak magnitudes. Once N is
% large enough, further increases in N do not significantly change the peak
% magnitudes, only making the peaks narrower and the spectrum smoother.
% For example, in the length 50 DFT, the peak magnitudes were in the teens
% and twenties, went up to the 50s and 60s for the length 160 DFT, and
% finally settle in the high 70s to low 80s range for the length 500 DFT,
```

% and remained unchanged from length 500 to the length 1000 DFT.

=====

=====

2. DFT and DTFT

=====

=====

```
% CREATE SUM OF SINUSOIDS
n = 0:59;
x1 = 0.75 + cos(pi*n/20) + cos(pi*n/15) + cos(pi*n + 2*pi/3);

% USE THE FOLLOWING FOR THE DTFT
w_DTFT = linspace(0, 2*pi-pi/10000, 10000);

% NOTE: USE THE FOLLOWING COMMENTED LINE FOR PLOTTING THE DFT ATOP THE DTFT
% (YOU NEED TO DEFINE w_DFT), THIS WILL MAKE THE PLOTS EASIER TO INTERPRET
% plot(w_DTFT,abs(X_DTFT));
% hold on; plot(w_DFT,abs(X_DFT),'.','markersize',10); hold off;

% -----
% 2(a) Create the X = DFT(x) function, which inputs a signal x and outputs
its DFT X with the
% same length
% -----
% Only need to modify function -- this area can be empty

% -----
% 2(b) Plot the magnitude of the DTFT of x[n]. Be sure to label axes with
proper units.
% -----
X_DTFT = DTFT(x1,w_DTFT);

figure
plot(w_DTFT,abs(X_DTFT))
xlabel('Normalized Frequency (rad/sample)')
ylabel('Magnitude')
title('Magnitude of DTFT of x[n]')

% -----
% 2(c) Plot the magnitude of the 60-length DFT of x[n] on top of the DTFT.
See the comment code
% on how to best plot the DFT on top of the DTFT plot. Note: w_DFT is the
```

```

same as w in the
% DFT function
% -----
X_DTFT = DTFT(x1,w_DTFT);
X_DFT = DFT(x1);
w_DFT = 2*pi*(0:length(x1)-1)/length(x1);

figure
plot(w_DTFT,abs(X_DTFT))
hold on
plot(w_DFT,abs(X_DFT),'.','markersize', 10)
hold off
xlabel('Normalized Frequency (rad/sample)')
ylabel('Magnitude')
title('DTFT and 60-Point DFT of x[n]')

% -----
% 2(d) Plot the magnitude of the 55-length DFT (i.e., remove the last 5
values from the original
% signal) of x[n] on top of the DTFT.
% -----
x2 = x1(1:55);
X_DFT = DFT(x2);
w_DFT = 2*pi*(0:length(x2)-1)/length(x2);

figure
plot(w_DTFT,abs(X_DTFT))
hold on
plot(w_DFT,abs(X_DFT),'.','markersize', 10)
hold off
xlabel('Normalized Frequency (rad/sample)')
ylabel('Magnitude')
title('DTFT and 55-Point DFT of x[n]')

% -----
% 2(e) Plot the magnitude of the 65-length DFT (i.e., add 5 zeros to the
original signal) of x[n] on
% top of the DTFT
% -----
x3 = [x1 zeros(1,5)];
X_DFT = DFT(x3);
w_DFT = 2*pi*(0:length(x3)-1)/length(x3);

figure
plot(w_DTFT,abs(X_DTFT))
hold on
plot(w_DFT,abs(X_DFT),'.','markersize', 10)
hold off
xlabel('Normalized Frequency (rad/sample)')
ylabel('Magnitude')
title('DTFT and 65-Point DFT of x[n]')

```

```

% -----
% 2(f) Plot the magnitude of the 200-length DFT of x[n] on top of the DTFT
% -----
x4 = [x1 zeros(1,140)];
X_DFT = DFT(x4);
w_DFT = 2*pi*(0:length(x4)-1)/length(x4);

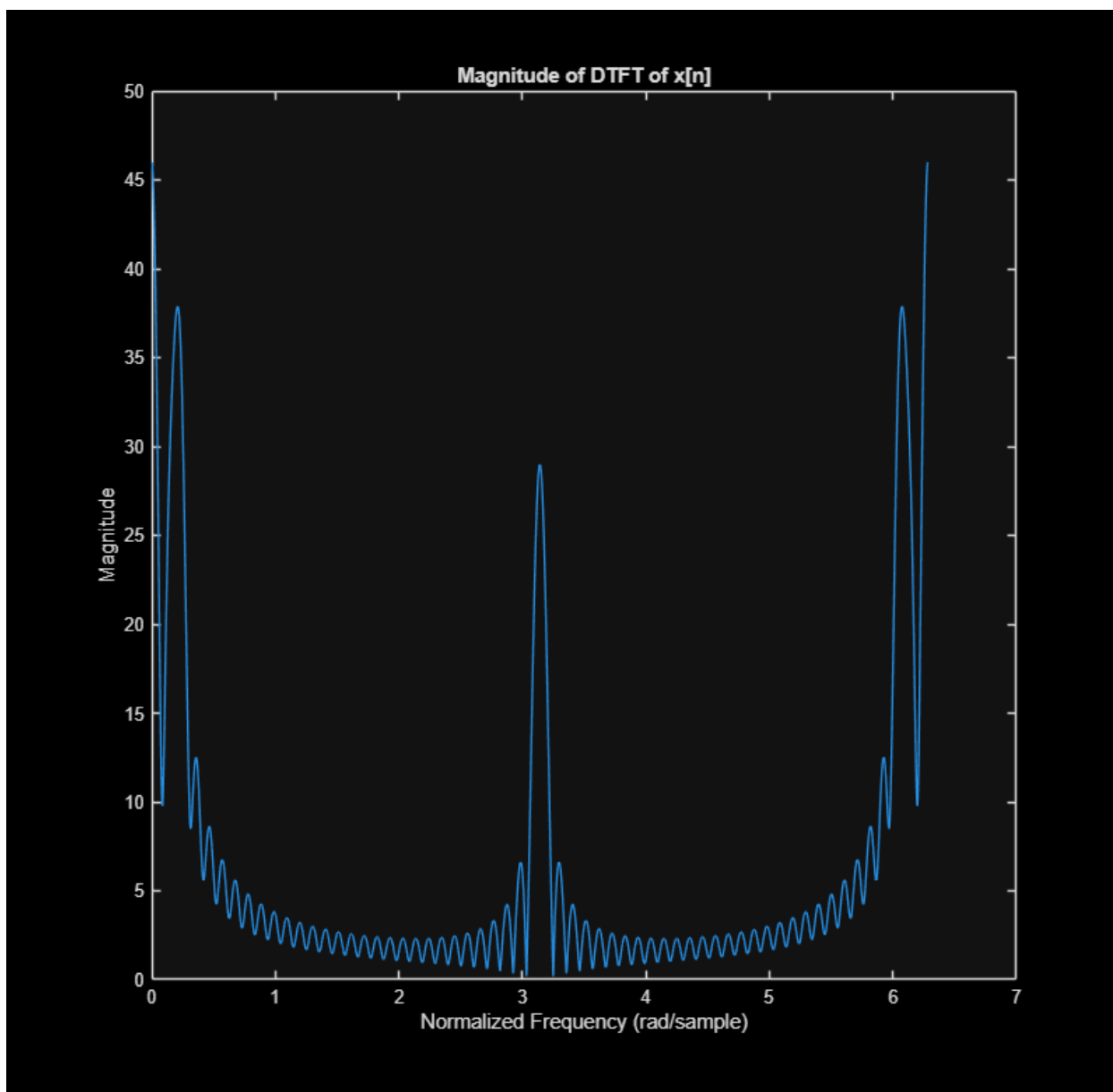
figure
plot(w_DTFT,abs(X_DTFT))
hold on
plot(w_DFT,abs(X_DFT),'.','markersize', 10)
hold off
xlabel('Normalized Frequency (rad/sample)')
ylabel('Magnitude')
title('DTFT and 200-Point DFT of x[n]')

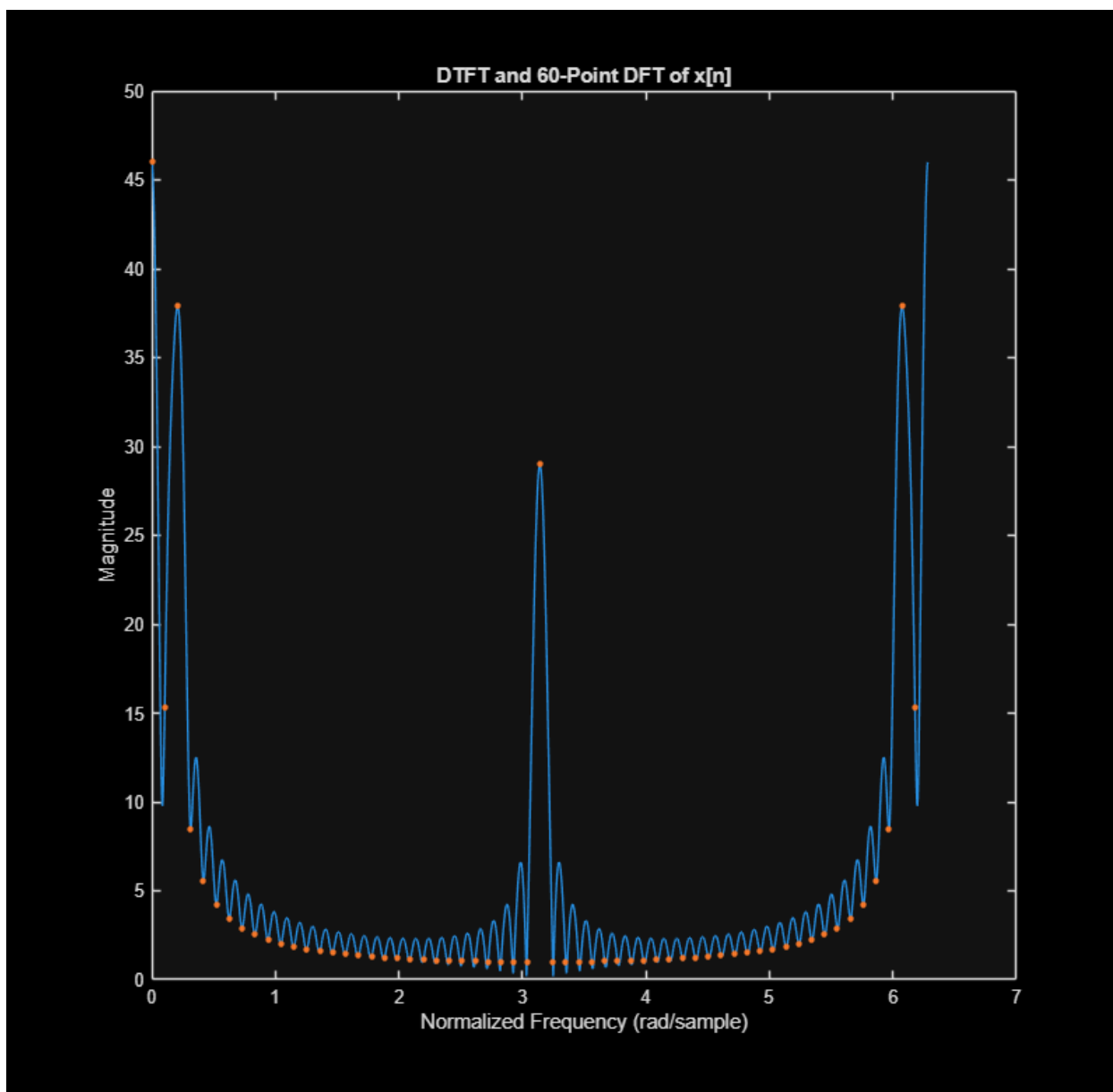
```

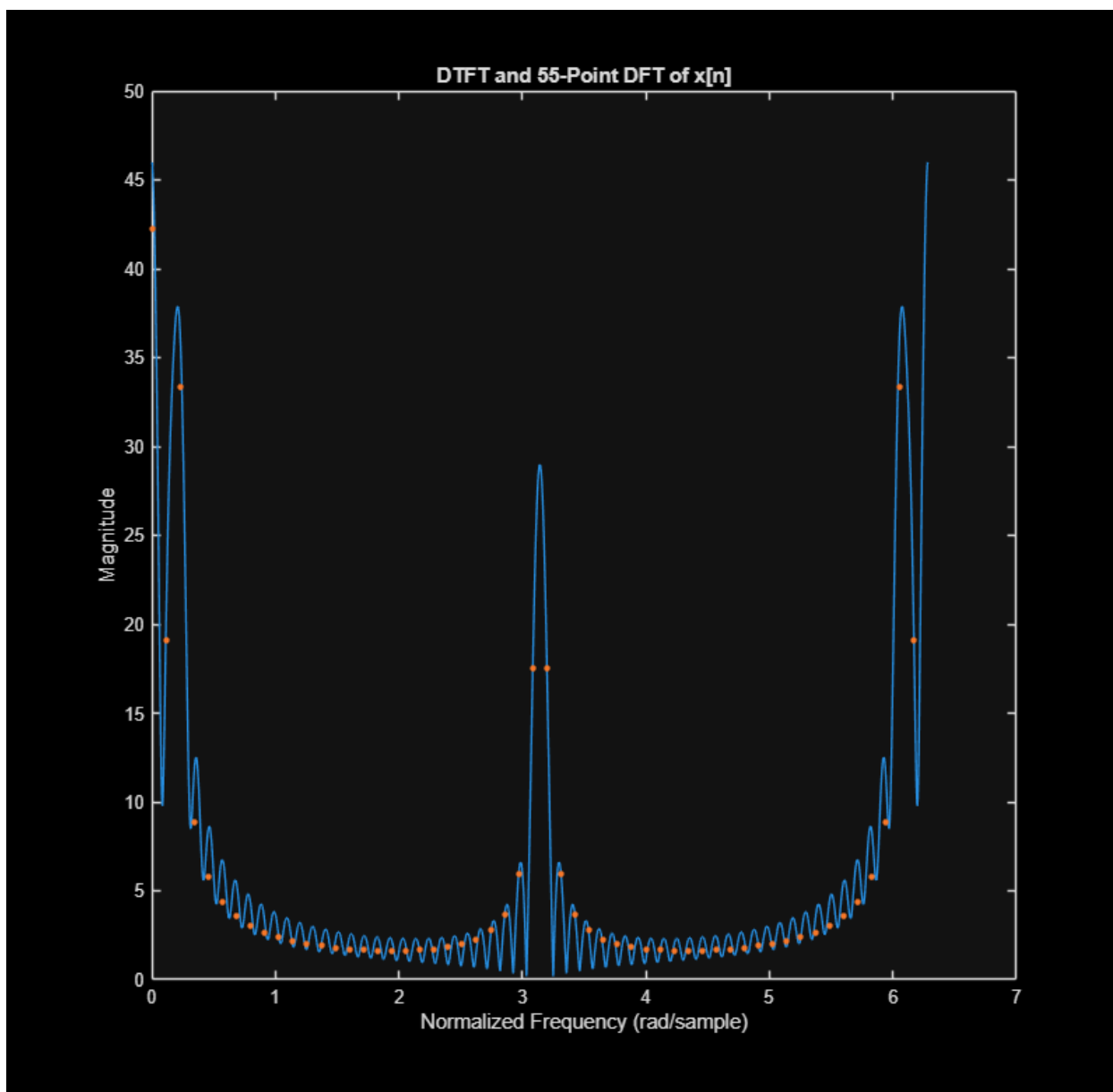
```

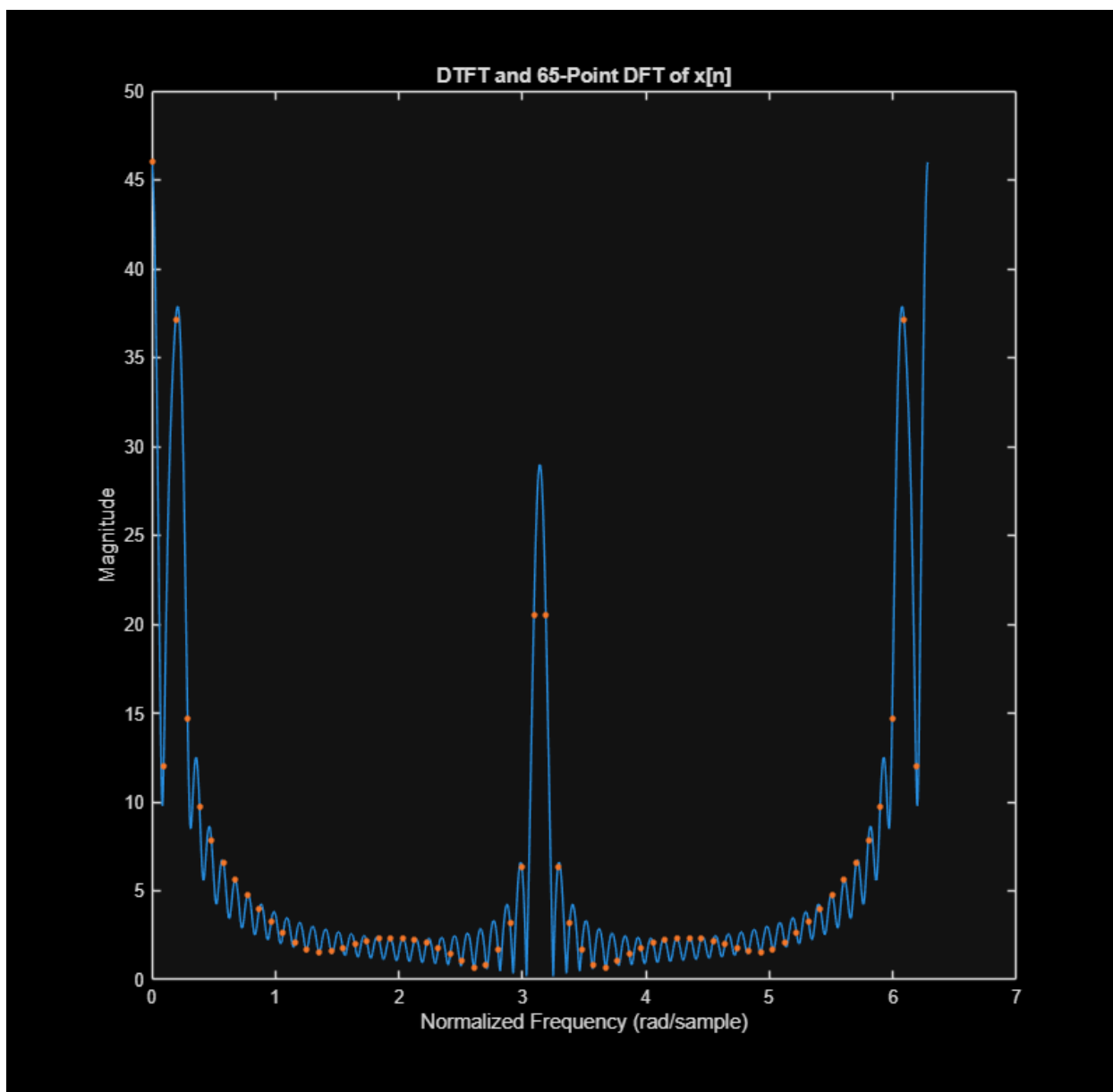
% -----
% 2(g) Based on the last several questions, what is the relationship
% between the DTFT and the DFT? Under what conditions will the theoretical
% DTFT and
% DFT have the same result?
% -----
% ANSWER QUESTION BELOW
%
% The DFT can be interpreted as a sampled version of the DTFT, where the
% DTFT is evaluated at N equally spaced frequency points over the interval
% from 0 to  $2\pi$ . As N increases, the DFT provides a denser sampling of the
% DTFT, making it appear closer to the continuous DTFT curve.
%
% The DFT most accurately captures the true peaks of the DTFT when the
% signal's frequency components align with the DFT frequency bins. When this
% occurs, there is no spectral leakage and the sampled DFT points coincide
% with the DTFT peaks. For example, the 60-length DFT captures some of the
% main peaks well, but does not perfectly align with all frequency
% components.
%
% In this case, choosing  $N = 120$  (or multiples of 120) would allow all
% frequency components, including  $\pi/20$ , to align exactly with DFT bins,
% since 120 is the least common multiple of the required periods. This would
% result in more accurate peak representation with minimal spectral leakage.
%
% In general, the DTFT and DFT will match at the sampled frequencies when
% the signal is fully captured within the observation window and its
% frequencies align with the DFT bin locations.
%

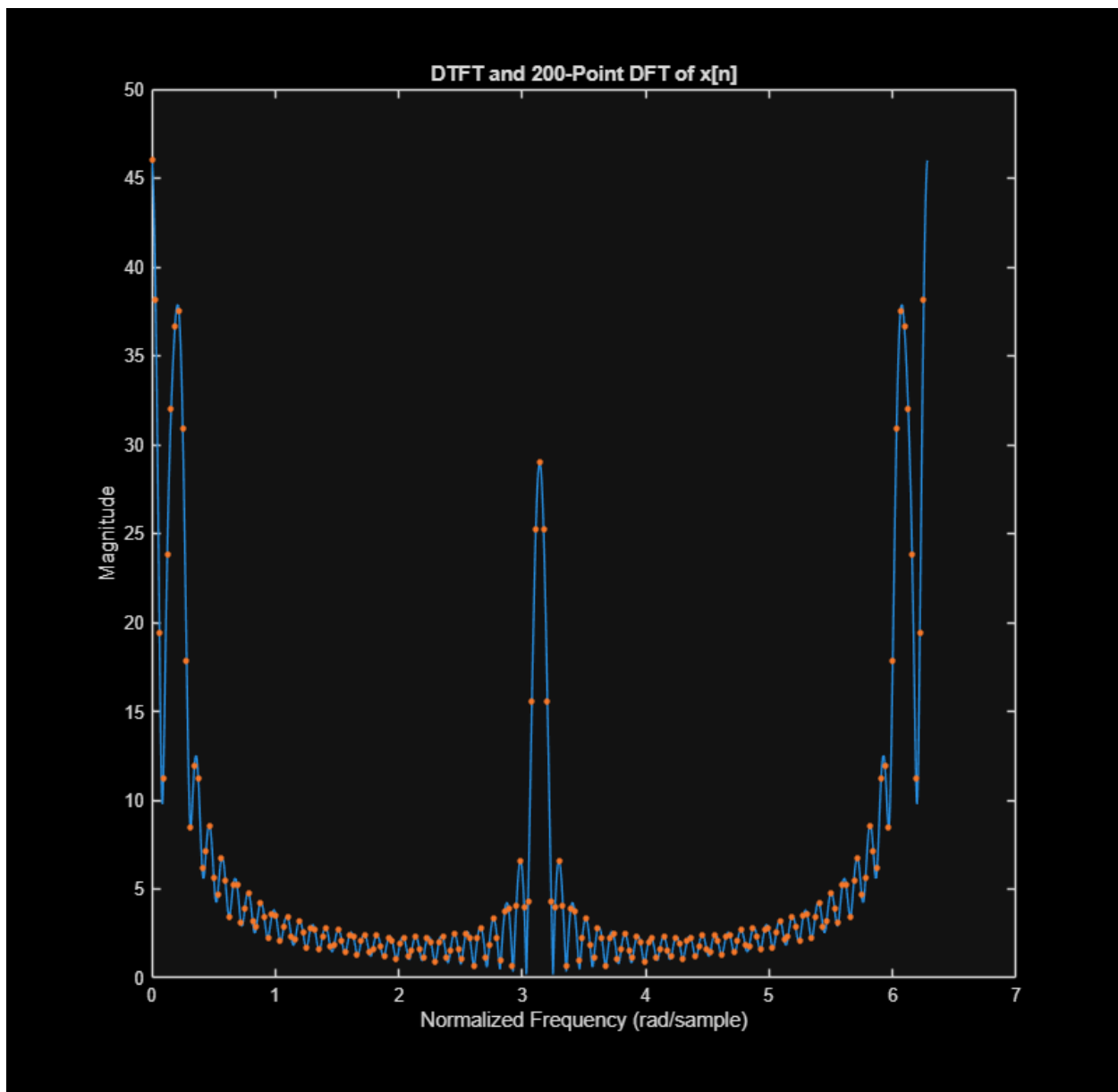
```











3. DFT and IDFT

----- 3(a) Create the $x = \text{IDFT}(X)$ function, which
inputs a DFT signal X and outputs its time-domain signal x with the same length.
----- Only need to modify function -- this area can be empty

```
% -----  
% 3(b) Plot (for  $0 \leq n \leq 99$ )  $x[n]$ . Use the stem function  
% -----  
n2 = 0:99;  
x2 = double((n2 >= 30) & (n2 < 65));  
  
figure  
stem(n2,x2)  
xlabel('n')  
ylabel('x[n]')  
title('x[n] = u[n-30] - u[n-65]')  
  
% -----  
% 3(c) Plot (for  $0 \leq n \leq 99$ )  $y[n] = x[n] * x[n]$ . Use the stem function.  
% -----  
y = conv(x2,x2);  
  
n3 = 0:length(y)-1;  
  
figure  
stem(n3,y)  
xlim([0 99])  
xlabel('n')  
ylabel('y[n]')  
title('y[n] = x[n] * x[n]')  
  
% I included a second plot because I did not know exactly how you wanted this  
% plotted.  
  
% n3 = 0:99;  
%  
% figure  
% stem(n3,y(1:100))  
% xlabel('n')  
% ylabel('y[n]')  
% title('y[n] = x[n] * x[n] (true only 0-99 plot)')
```

```

% -----
% 3(d) Plot (for  $0 \leq n \leq 99$ )  $\text{IDFT}\{\text{DFT}\{x[n]\}\text{DFT}\{x[n]\}\}$ . Use  $N = 100$  length
DFT and IDFT
% functions. Use the stem function
% -----
x_pad = [x2 zeros(1,100-length(x2))];

X = DFT(x_pad);
Y = X .* X;

y_idft = IDFT(Y);

n4 = 0:99;

figure
% stem(n4,y_idft(1:100))
stem(n4,real(y_idft(1:100)))
xlabel('n')
ylabel('y[n]')
title('IDFT{DFT{x[n]}DFT{x[n]}} with N = 100')

% -----
% 3(e) Plot (for  $0 \leq n \leq 99$ )  $\text{IDFT}\{\text{DFT}\{x[n]\}\text{DFT}\{x[n]\}\}$ . Use  $N = 200$  length
DFT and IDFT
% functions. Use the stem function.
% -----
x_pad2 = [x2 zeros(1,200-length(x2))];

X2 = DFT(x_pad2);
Y2 = X2 .* X2;

y_idft2 = IDFT(Y2);

n5 = 0:99;

figure
stem(n5, real(y_idft2(1:100)))
xlabel('n')
ylabel('y[n]')
title('IDFT{DFT{x[n]}DFT{x[n]}} with N = 200')

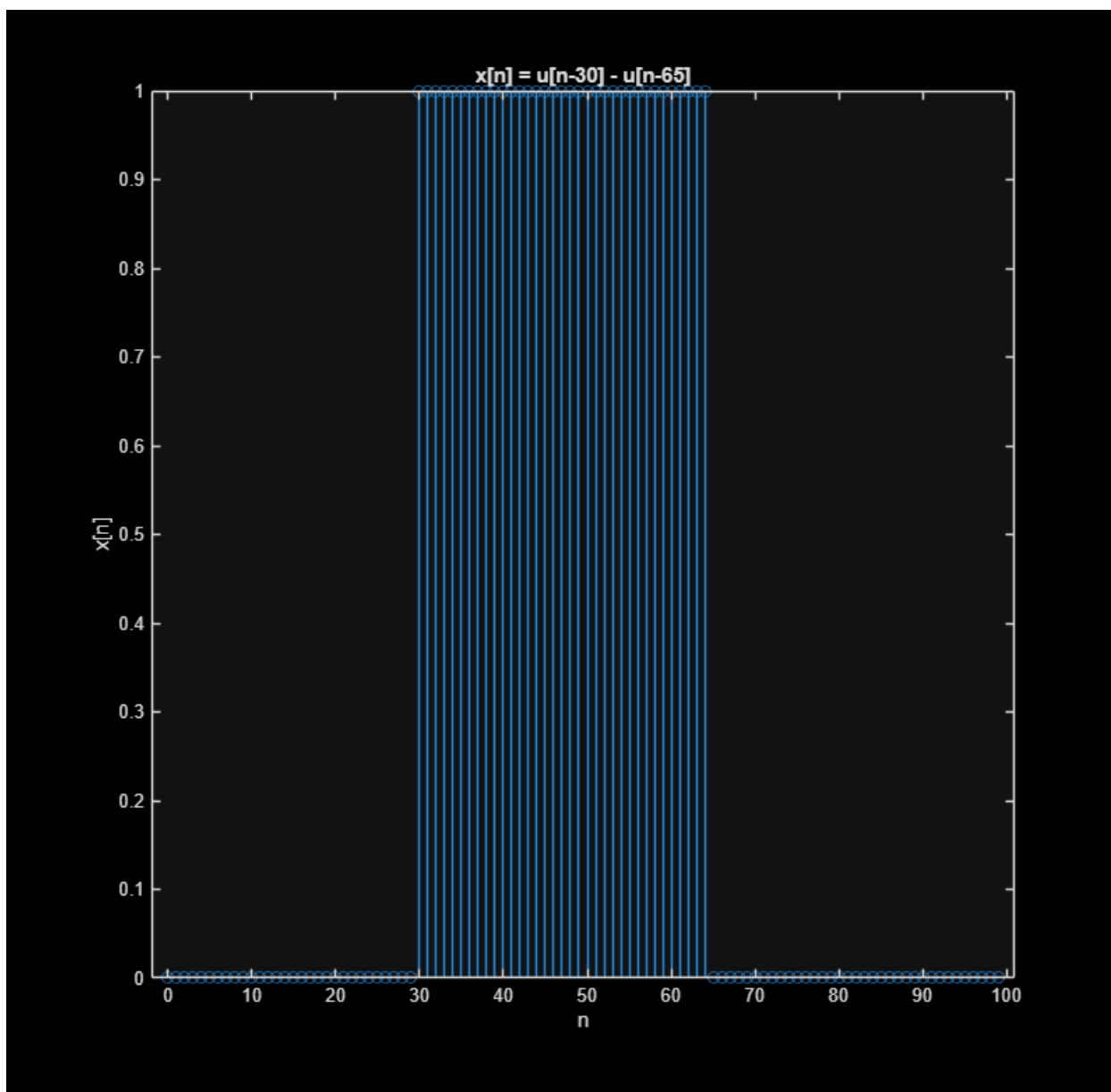
% plotting to see what it would be if I plot 200 samples

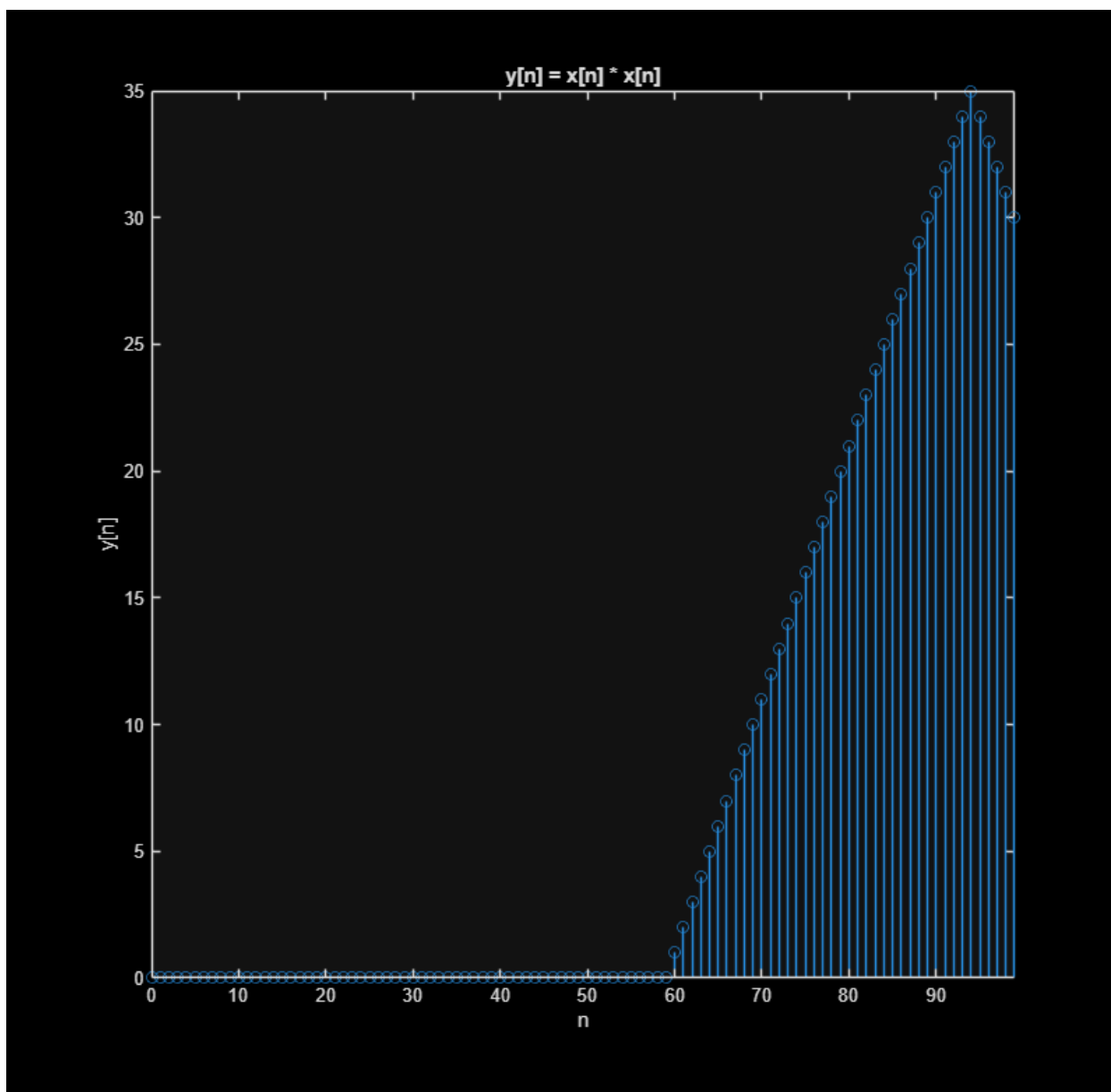
n_full = 0:199;

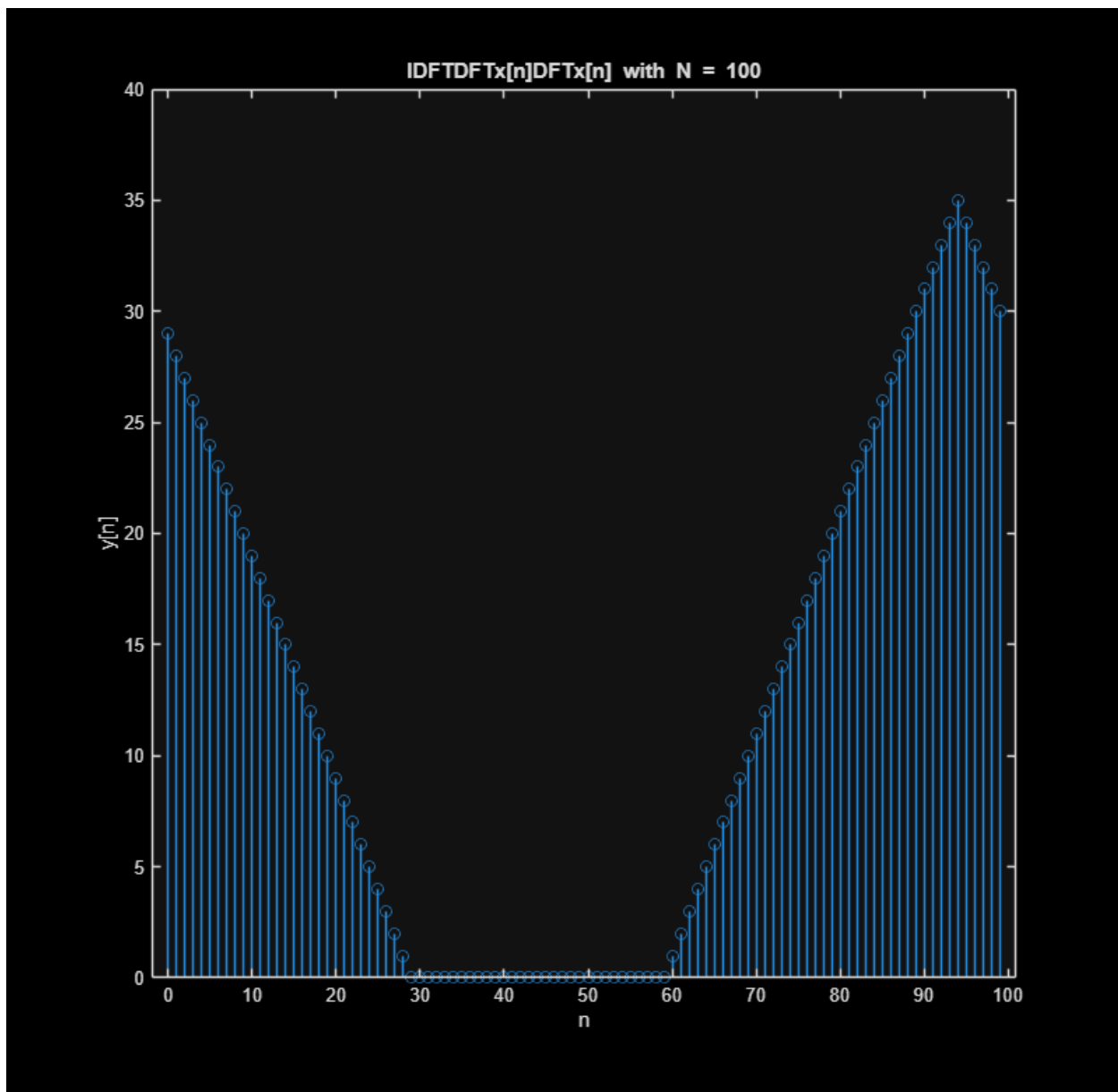
figure
stem(n_full, real(y_idft2))
xlabel('n')
ylabel('y[n]')
title('Full result with N = 200 (Extra Plot)')

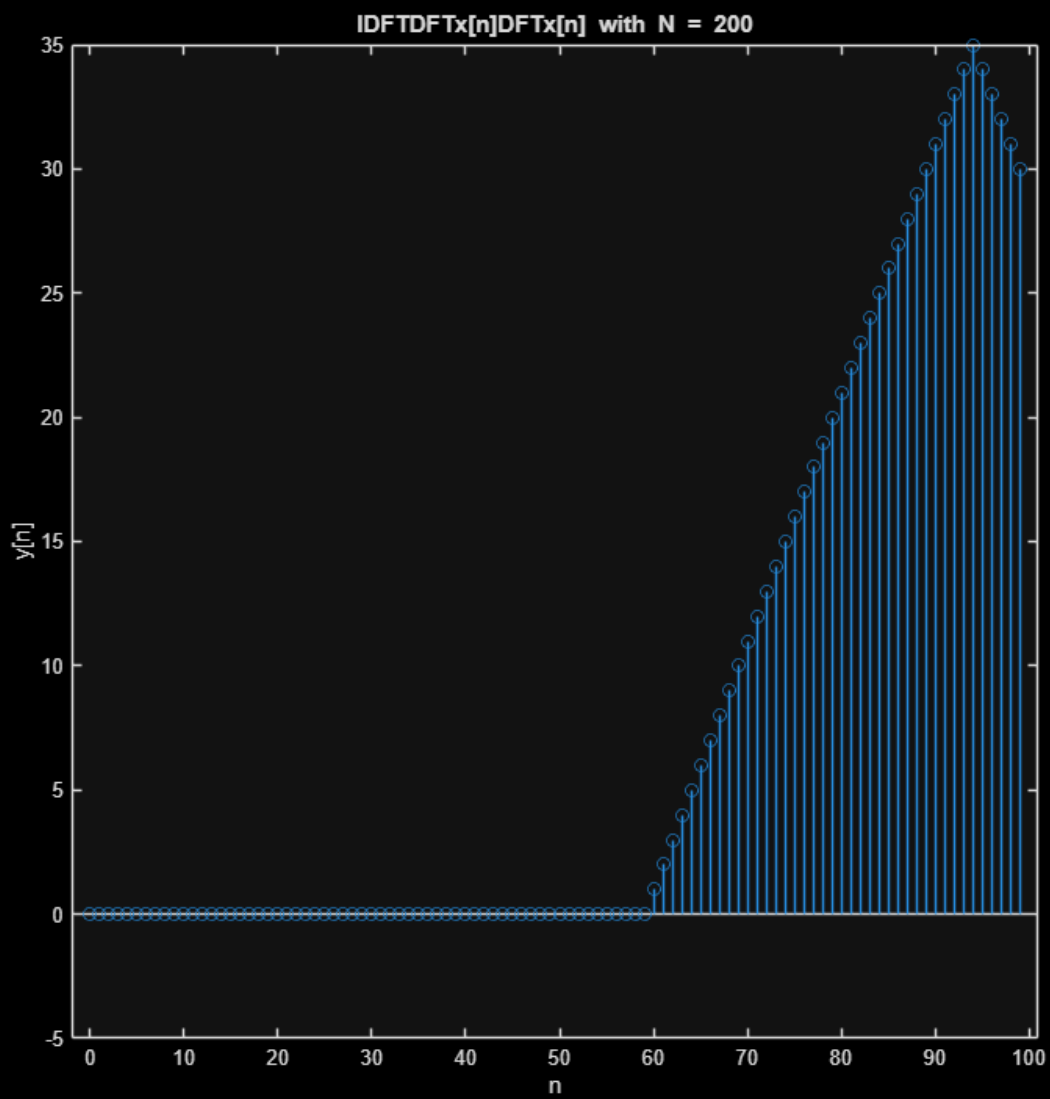
```

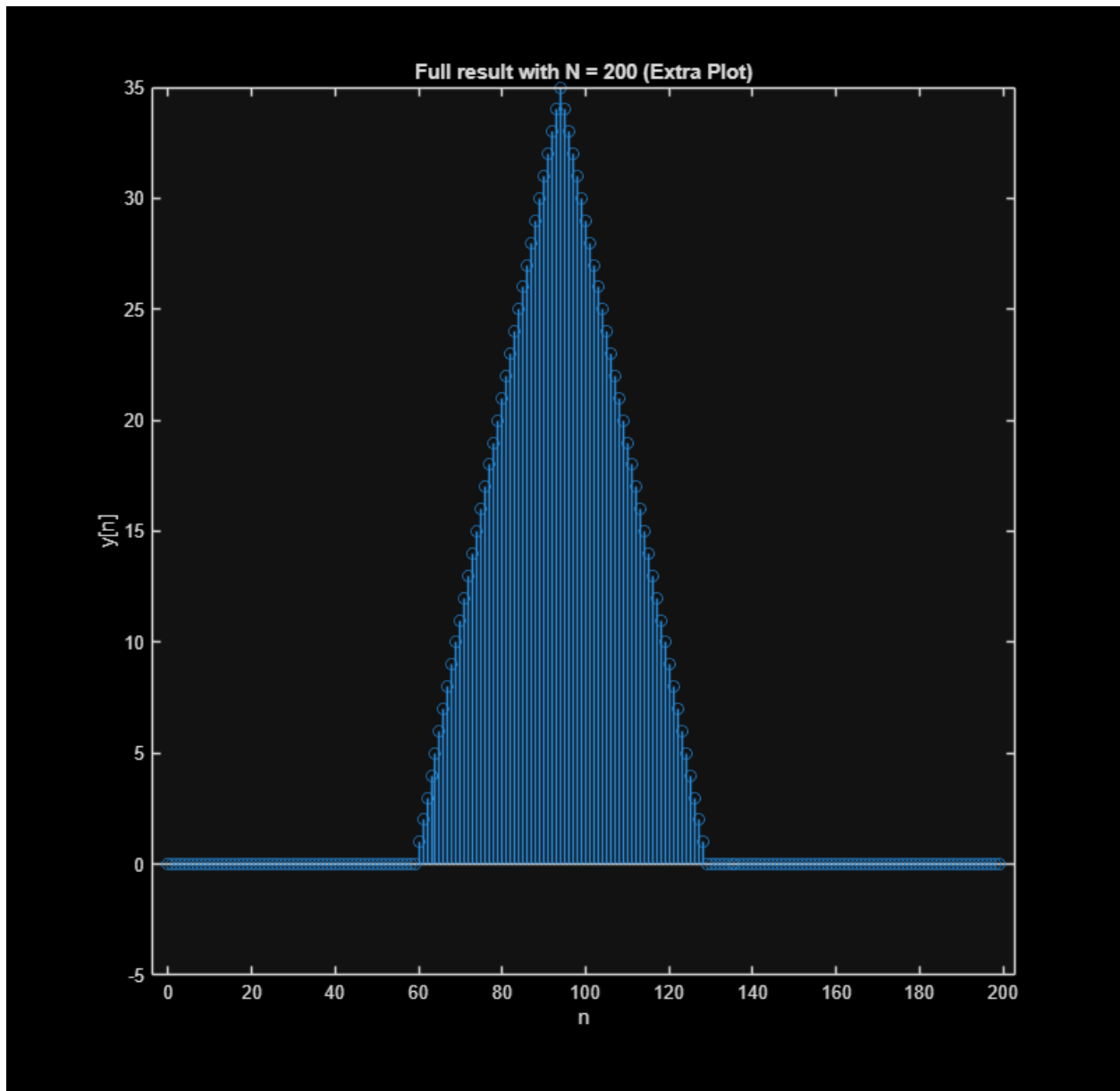
```
% -----
% 3(f) Answer in your comments: What is the difference in the last two
% solutions? Why does
% the difference exist?
% -----
% ANSWER QUESTION BELOW
%
% The difference between the results in parts (d) and (e) is that part (d)
% produces circular convolution with wraparound, while part (e) matches the
% linear convolution over the plotted range.
%
% In part (d),  $N = 100$  is not large enough to contain the full linear
% convolution, whose length extends to  $n = 128$ . As a result, the values that
% exceed  $n = 99$  wrap around to the beginning of the signal, causing the
% triangular shape to split and appear on both ends of the plot.
%
% In part (e),  $N = 200$  is large enough to contain the entire linear
% convolution without overlap. Therefore, no wraparound occurs, and the
% result matches the true linear convolution for  $0 \leq n \leq 99$ .
% (You can see this with the extra 200 point plot, since the 100 point plot
% gets cut off).
```











=====

=====

4. DFT AND FFT

=====

=====

Choose a song at least 3 minutes long to use in this problem, and include it in your submission. Load it into MATLAB using audioread. Note that most audio files will be stereo, so you need to make sure that you only use

one column of audio data for this part of the lab This site has a large archive of free music that you can choose from:
<https://freemusicarchive.org/static>

```
[x3,fs] = audioread('John Harrison with the Wichita State University Chamber  
Players - Spring Mvt 1 Allegro.mp3');  
x3 = x3(:,1);
```

```
length(x3)/fs
```

```
x4 = x3(1:10000);  
n6 = 0:length(x4)-1;
```

```
% -----  
% 4(a) Plot the magnitude of the DFT of only the first 10000 samples of the  
audio and display (using  
% the disp function) the output of tic and toc (read help tic and help toc)  
to measure the  
% length of time it takes to compute the DFT, in seconds.  
% -----
```

```
x4 = x3(1:10000);
```

```
tic  
X_DFT_audio = DFT(x4);  
t_dft = toc;  
disp(['DFT time: ', num2str(t_dft), ' seconds'])
```

```
N_audio = length(x4);  
w_audio = -pi:2*pi/N_audio:pi-2*pi/N_audio;
```

```
figure  
plot(w_audio, fftshift(abs(X_DFT_audio)))  
xlabel('Normalized Frequency (rad/sample)')  
ylabel('Magnitude')  
title('Magnitude of DFT of First 10000 Audio Samples')  
xlim([-pi pi])
```

```
% -----  
% 4(b) Plot the magnitude of the FFT of only the first 10000 samples of  
the audio and display  
% (using the disp function) the output of tic and toc to measure the length  
of time it takes  
% to compute the FFT, in seconds.  
% -----
```

```
tic  
X_FFT_audio = fft(x4);  
t_fft = toc;  
disp(['FFT time: ', num2str(t_fft), ' seconds'])
```

```
N_audio = length(x4);  
w_audio = -pi:2*pi/N_audio:pi-2*pi/N_audio;
```

```

figure
plot(w_audio, fftshift(abs(X_FFT_audio)))
xlabel('Normalized Frequency (rad/sample)')
ylabel('Magnitude')
title('Magnitude of FFT of First 10000 Audio Samples')
xlim([-pi pi])

```

```

% -----
% 4(c) Are there differences in the magnitude results? If so, why?
% -----

```

```

% ANSWER QUESTION BELOW

```

```

%
% The magnitude results from the DFT and FFT are essentially the same
% (I was not able to detect any differences looking over them).
% Any very small differences are due to numerical precision and computation,
% since both methods are computing the same discrete Fourier transform.
% The FFT is simply a more efficient algorithm for computing the DFT.
%

```

```

% -----
% 4(d) How much faster is the FFT algorithm compared with the DFT
% in this scenario?
% -----

```

```

% ANSWER QUESTION BELOW

```

```

%
% In this scenario, the FFT is much faster than the DFT. Using the measured
% times, the speedup is  $1.4638 / 0.0003509 \approx 4171.558849$ , so the FFT is about
% 4172 times faster than the DFT for the first 10000 audio samples.
% (These numbers were from the first time running the function; they will
% vary each time it is run. In the few runs I did the FFT
% usually performed around 3500 to 7000 times faster than the DFT.)

```

```

% -----
% 4(e)
% Display (using the disp function) the output of tic and toc to measure the
length of time
% it takes to compute the FFT of the entire song, in seconds. Using DFT on
an entire 3 minute
% audio (sampled at 44.1 kHz) should take approximately 500 hours, so let's
not do that one.
% -----

```

```

tic
X_full_fft = fft(x3);
t_full_fft = toc;
disp(['Full-song FFT time: ', num2str(t_full_fft), ' seconds'])

```

```

ans =

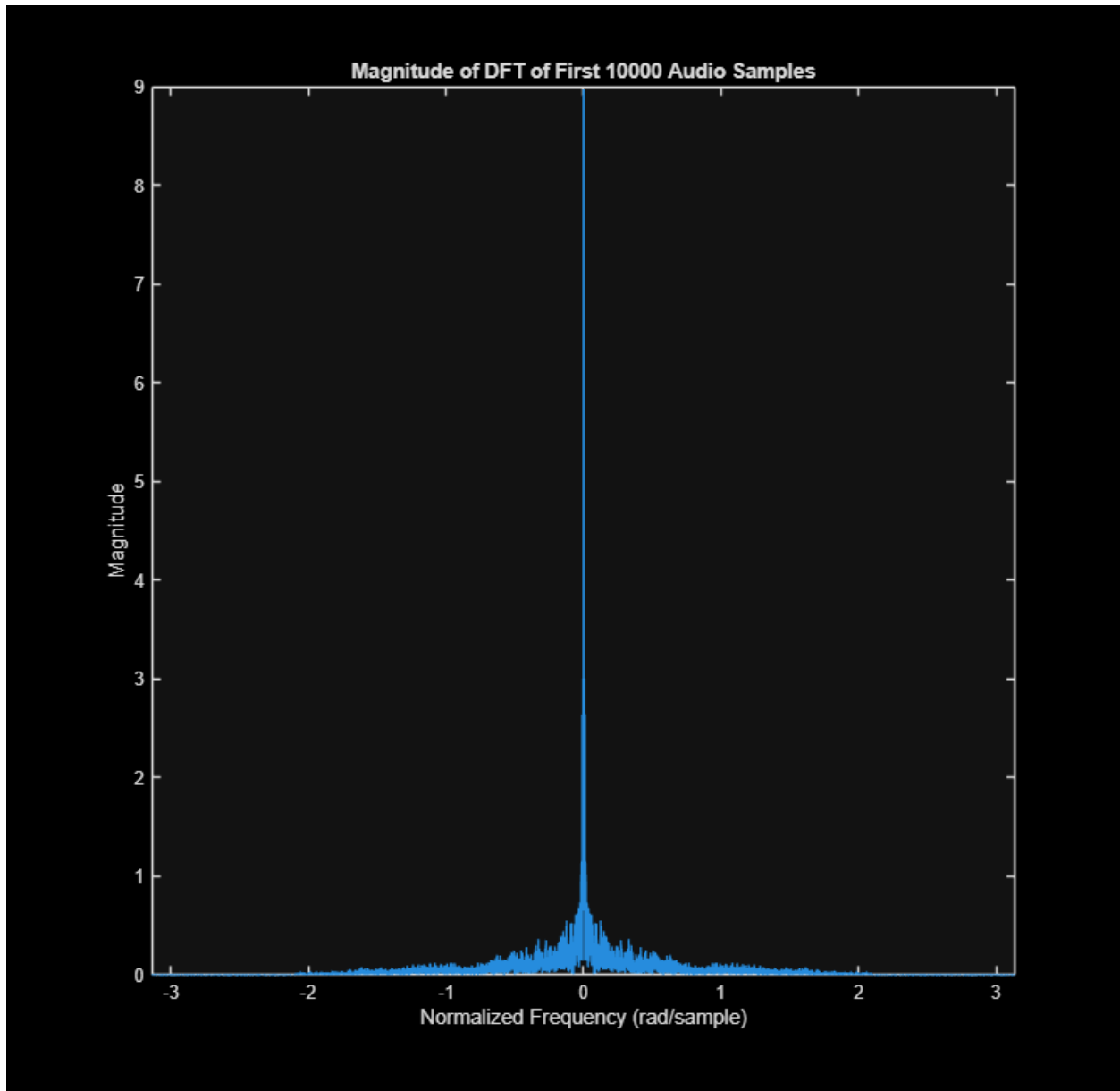
```

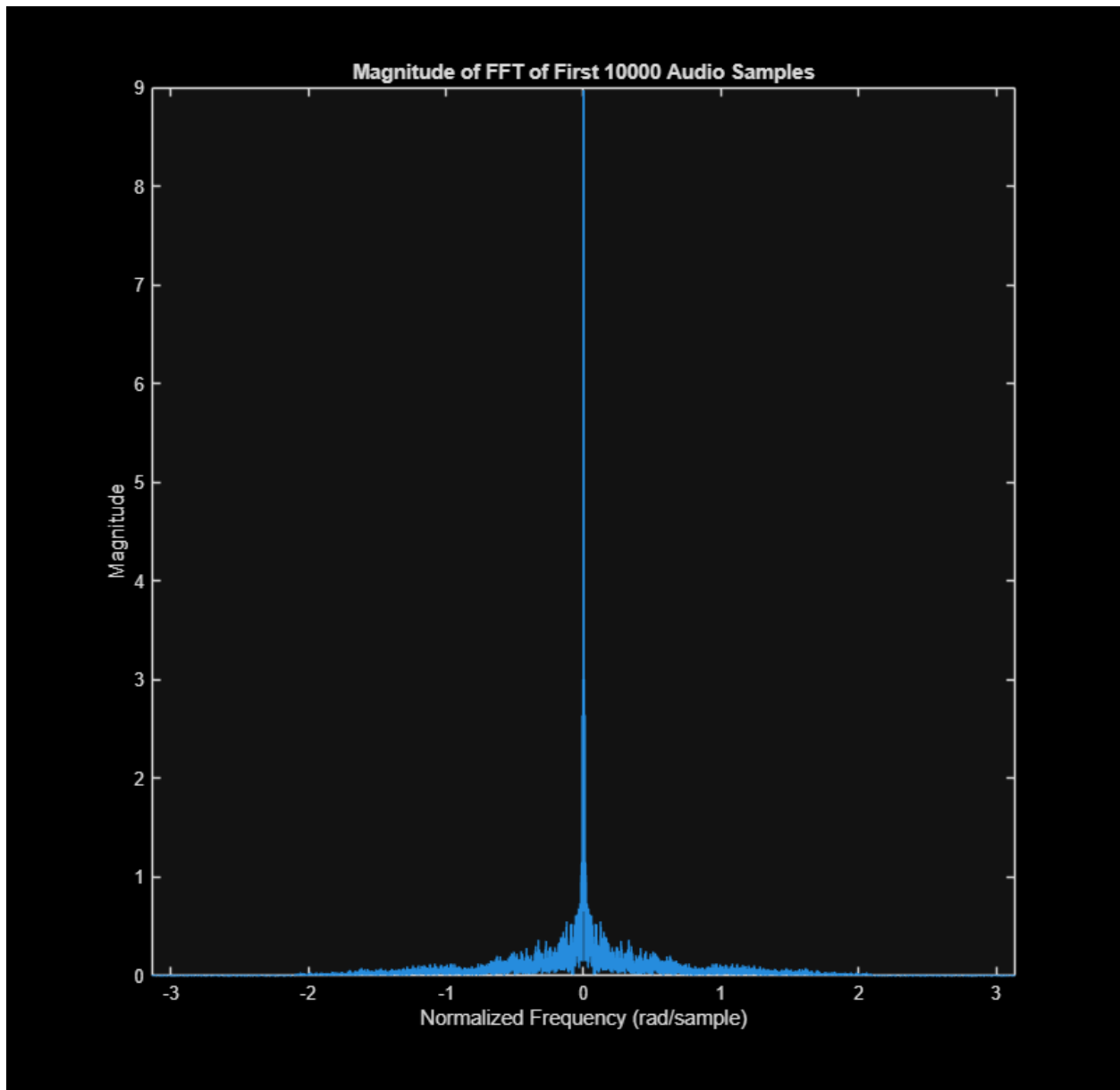
215.3170

DFT time: 1.4295 seconds

FFT time: 0.0004079 seconds

Full-song FFT time: 0.14736 seconds





Functions provided for the lab

```
function H = DTFT(x,w)
% DTFT(X,W)  compute the Discrete-time Fourier Transform of signal X
% across frequencies defined by W.

H = zeros(1, length(w));
for nn = 1:length(x)
    H = H + x(nn).*exp(-1j*w.*(nn-1));
end

end

function X = DFT(x)
```

```
% DFT(x)  compute the N-point Discrete Fourier Transform of signal x
% Where N is the length of signal x
    N = length(x);
    w = 2*pi*(0:N-1)/N;
    X = zeros(1, length(w));
    for nn = 1:length(x)
        X = X + x(nn).*exp(-1j*w.*(nn-1));
    end
end

function x = IDFT(X)
% IDFT(x)  compute the N-point Inverse Discrete Fourier Transform of signal
% X where N is the length of signal X
    N = length(X);
    w = 2*pi*(0:N-1)/N;
    x = zeros(1, length(w));
    for nn = 1:length(x)
        x = x + X(nn).*exp(1j*w.*(nn-1));
    end
    x = x/N;
end
```

Published with MATLAB® R2025b